



PROITBRIDGE
Strive For Better Future



» AGENTIC « AI



From Prompts to Autonomy:
the complete playbook for agents that
think, act, remember, and collaborate.



AI Coach John
Follow for More

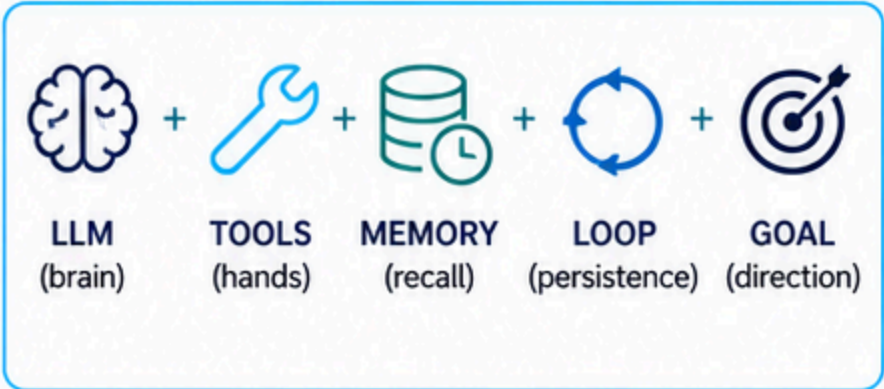


FOUNDATIONS: WHAT IS AN AGENT?

1. AGENT FOUNDATIONS | Why LLMs alone aren't enough.

1. THE CORE IDEA

A plain LLM answers. An agent achieves.
The difference is a loop with feedback from the real world.



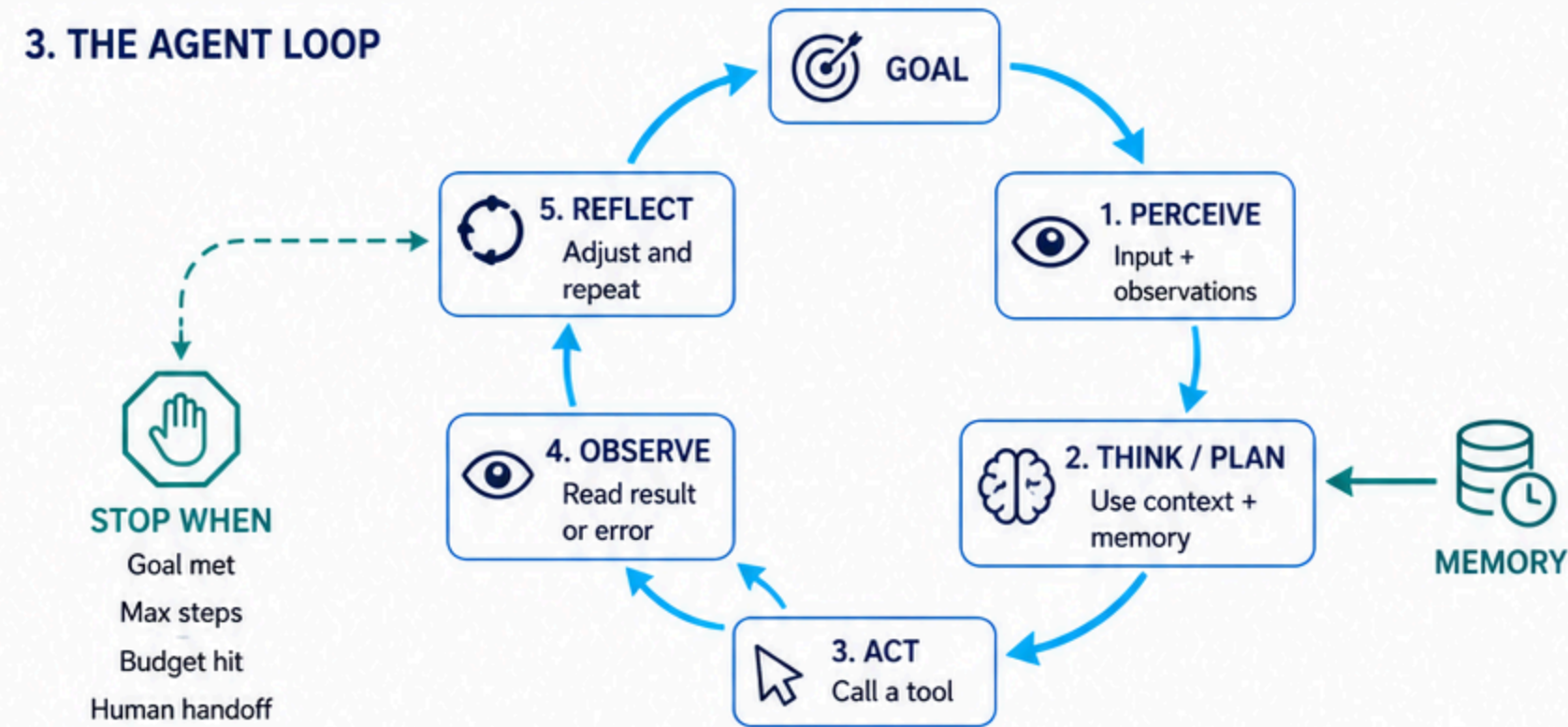
2. CONTEXT: WHY AGENTS EMERGED

- LLMs are frozen in time → Tools provide live data
- LLMs cannot act → Tools create effects in the world
- LLMs forget → Memory provides continuity
- One-shot answers fail on multi-step work → Loops enable recovery



Example: Research a market, compare options, draft a report, verify sources, and revise after errors.

3. THE AGENT LOOP



4. WORKFLOW vs AGENT

	WORKFLOW	AGENT
Control flow	Developer-defined	LLM-decided at runtime
Path	Fixed DAG	Dynamic, branching
Cost / latency	Predictable	Variable
Debuggability	Easier	Harder
Best for	Known, repeatable tasks	Ambiguous, open-ended tasks



RULE OF THUMB

Start with a workflow. Add agency only where the path genuinely cannot be predicted.

5. LEVELS OF AUTONOMY

- L0** Prompt → Response | No tools
- L1** Single tool call | RAG or search
- L2** Fixed chain | Workflow
- L3** Router | LLM picks branch
- L4** Autonomous loop | Self-directed steps
- L5** Multi-agent org | Agents manage agents



MORE CAPABILITY ↑
LESS PREDICTABILITY ↓

6. WHEN NOT TO BUILD AN AGENT

- Deterministic logic
- Sub-100 ms latency
- Zero error tolerance without review
- Simple Q&A
- Very high volume with a tight cost budget
- Regulated or safety-critical actions without human approval



TAKEAWAY

Agents trade predictability for adaptability.
Spend that trade deliberately.





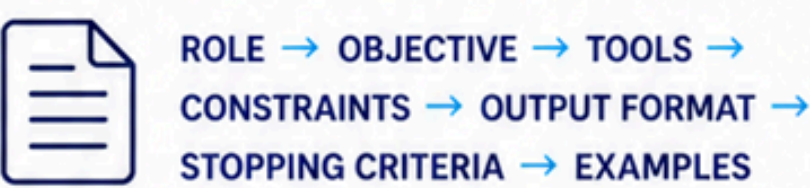
ANATOMY OF AN AGENT

2. AGENT ARCHITECTURE | DEEP DIVE

1. THE COMPONENT STACK

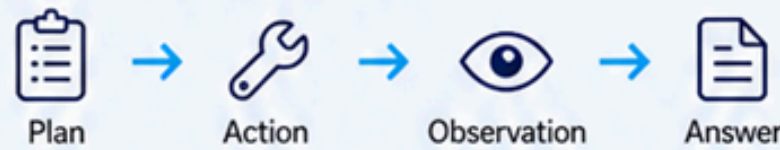


2. THE AGENT PROMPT SKELETON



Under TOOLS: Define when to use each.
Under CONSTRAINTS: Include forbidden actions.

WORKED TRAJECTORY



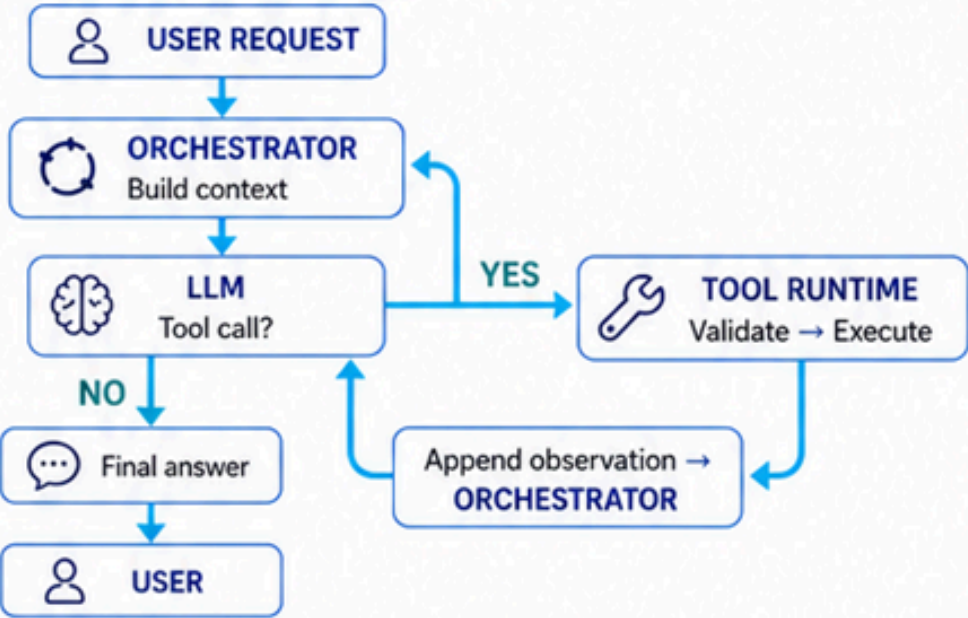
Example: Find current sales data → Query CRM →
Validate returned records → Summarize with sources



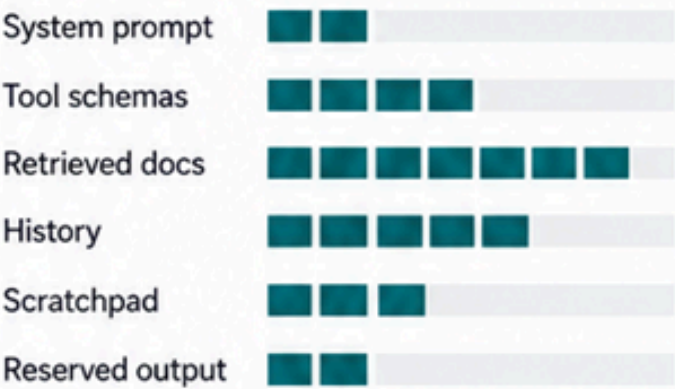
DESIGN TIP

Show the model a full worked trajectory.
One good example can beat ten rules.

3. RUNTIME EXECUTION FLOW



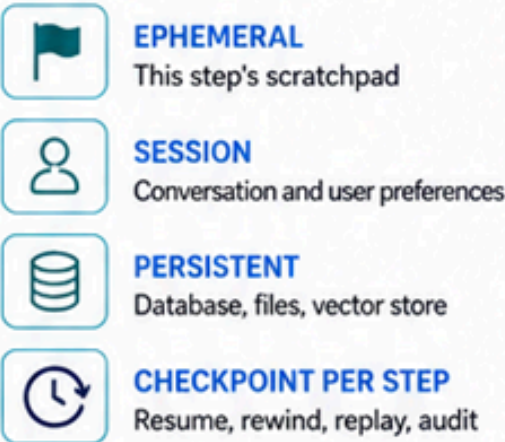
4. CONTEXT WINDOW BUDGET



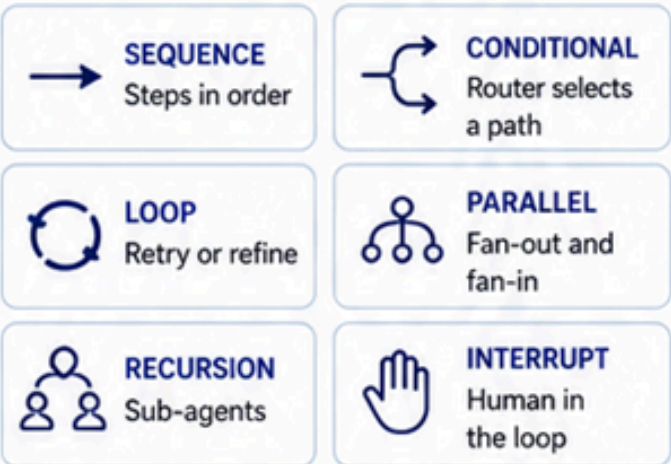
BUDGET TIP

Twenty tool schemas can consume 5,000+ tokens before any work begins. Prune aggressively.

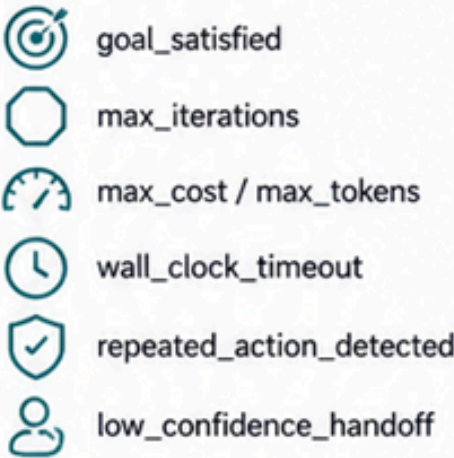
5. STATE & CHECKPOINTING



6. CONTROL-FLOW PRIMITIVES



7. TERMINATION CONDITIONS



TAKEAWAY

Every loop needs a brake.

Unbounded agents can turn small mistakes into large costs.
Set limits before execution.



REASONING & PLANNING PATTERNS

3. HOW AGENTS THINK

1. CHAIN-OF-THOUGHT (CoT)



- Linear reasoning before answering.
- No tools and low cost, but it cannot recover once the reasoning goes wrong.
- Best for: Short math, logic, and structured analysis.

2. ReAct | REASON + ACT

THE WORKHORSE

ILLUSTRATIVE EXAMPLE

- Reasoning note:** I need the current price of NVDA
- Action:** search("NVDA stock price")
- Observation:** \$184.20
- Reasoning note:** Now compute P/E using EPS
- Action:** calculator("184.20 / 3.10")
- Observation:** 59.4
- Final answer:** NVDA trades at a P/E of about 59
Sample figures only

KEY INSIGHT



Interleaving reasoning with real observations is one of the strongest ways to reduce hallucinations.

3. PLAN-AND-EXECUTE



On failure or new information:



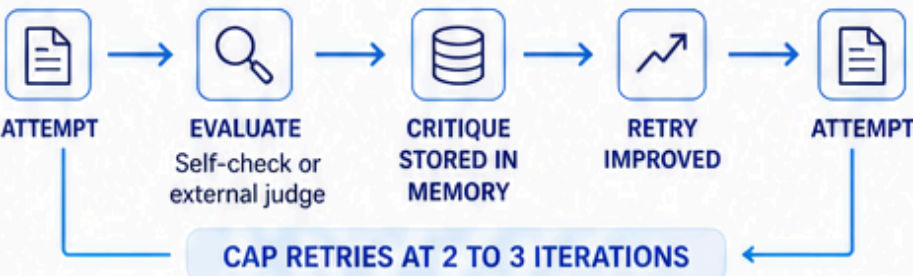
ADVANTAGES

- Lower execution cost
- Better long-horizon coherence

LIMITATION

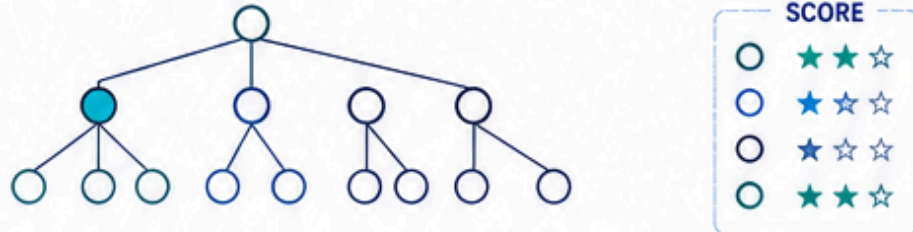
Can become brittle when the environment changes mid-plan.

4. REFLEXION / SELF-CRITIQUE



5. TREE / GRAPH OF THOUGHTS

Branch candidate paths -> Score -> Prune -> Expand the best



High cost. Best for puzzles, planning, and code search.

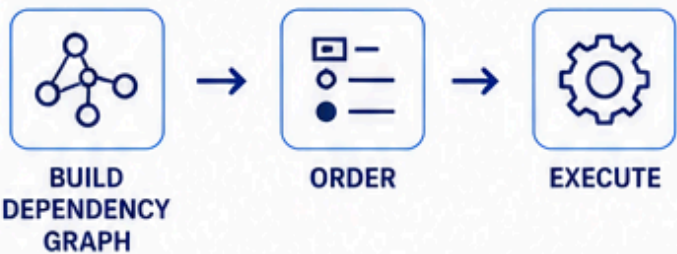
6. PATTERN COMPARISON

Pattern	Tools	Cost	Recovery	Best for
CoT	No	\$	None	Math, logic
Self-Consistency	No	\$\$	Limited	Accuracy boost
ReAct	Yes	\$\$	Strong	General agents
Plan-Execute	Yes	\$\$	Replan	Long tasks
ReWOO	Yes	\$	Limited	Low latency
Reflexion	Yes	\$\$\$	Strong	Code, writing
ToT / LATS	Sometimes	\$\$\$\$	Strong	Search, puzzles

7. TASK DECOMPOSITION



Then:



TAKEAWAY

Match the pattern to the task.

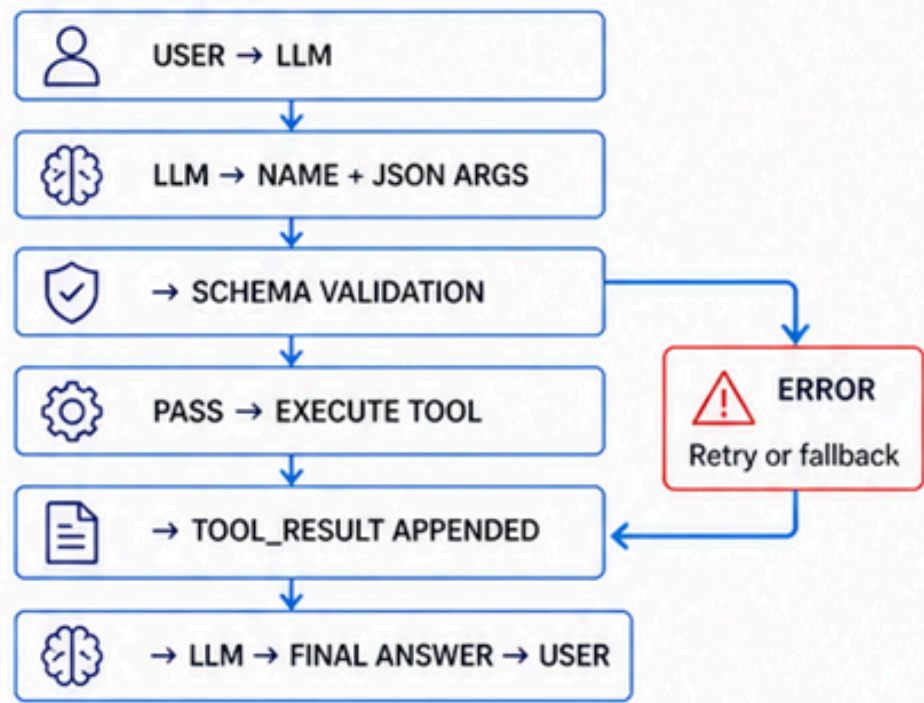
Reflexion on a lookup query wastes effort.
CoT on a 30-step job invites failure.



TOOLS & FUNCTION CALLING

4. TOOLS | GIVING THE AGENT HANDS

1. THE TOOL-CALL CYCLE



2. ANATOMY OF A GOOD TOOL DEFINITION

NAME	Use verb_noun
DESCRIPTION	State when to use it and when not to
PARAMETERS	Typed JSON Schema with examples
REQUIRED	Mark essential fields
RETURNS	Define the result shape
ERRORS	Enumerate expected failures
DESIGN TIP The description is a prompt. Write it for the model, not only as a human API document.	
MINI EXAMPLE name: search_orders required: customer_id returns: order_id, status	

3. TOOL CATEGORIES



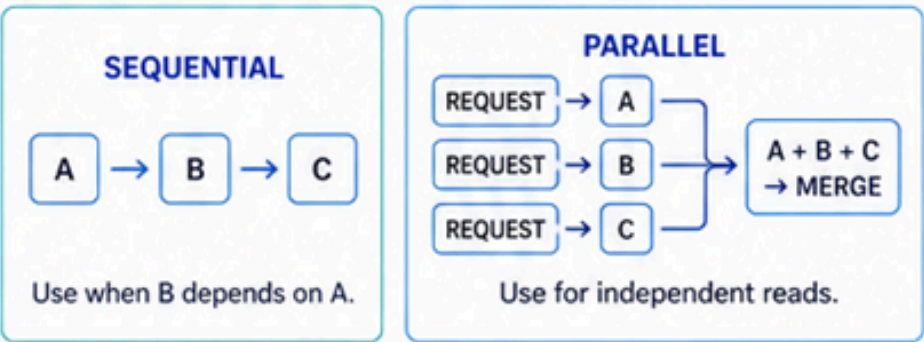
4. READ vs WRITE



5. TOOL SELECTION PROBLEMS → FIXES

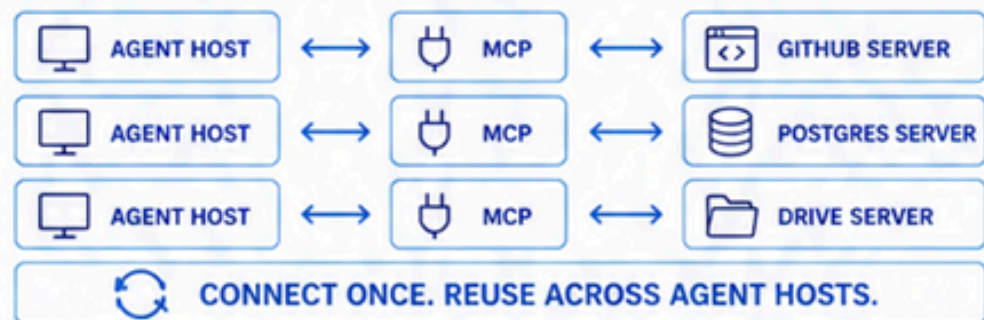
Too many tools	→	Keep 10 to 15 active; retrieve others dynamically
Overlapping tools	→	Namespace tools and sharpen descriptions
Wrong arguments	→	Use strict schemas, enums, and defaults
Tool spam loops	→	Deduplicate repeated calls and cap per-tool calls

6. PARALLEL vs SEQUENTIAL

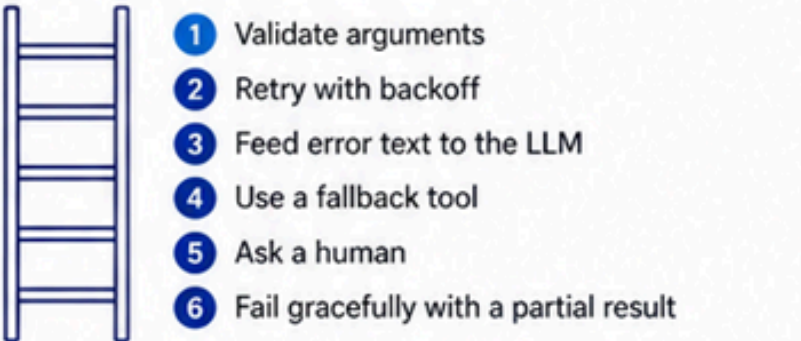


7. MCP | MODEL CONTEXT PROTOCOL

An open standard that lets an agent connect to tools and data sources.



8. ERROR-HANDLING LADDER



TAKEAWAY

Tools turn answers into actions.
Strong schemas, scoped permissions, and bounded retries make those actions reliable.



MEMORY & CONTEXT ENGINEERING

5. MEMORY | GIVING THE AGENT A PAST

1. MEMORY TYPES



SHORT-TERM | WORKING
Current context window and scratchpad



LONG-TERM | SEMANTIC
Facts, preferences, and documents stored in vector or graph systems

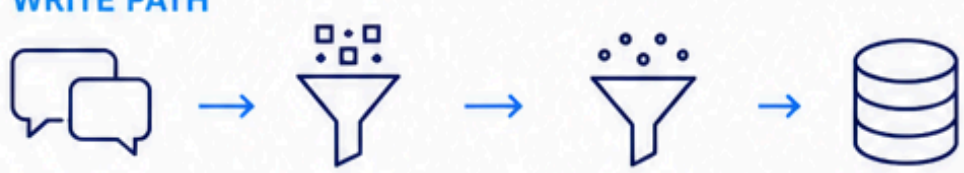


EPISODIC
Past runs and trajectories
Example: "Last time this failed"



PROCEDURAL
Learned skills, prompts, and cached plans

2. MEMORY WRITE / READ FLOW

WRITE PATH


CONVERSATION → EXTRACT (Summarize and score importance) → DEDUPLICATE → STORE (Vector database, SQL, or graph)

READ PATH


NEW TURN → EMBED QUERY → RETRIEVE TOP K → RERANK → INJECT INTO CONTEXT

3. CONTEXT MANAGEMENT STRATEGIES



WINDOW
Keep the last N turns



SUMMARIZE
Roll old turns into a running digest



OFFLOAD
Write large artifacts to files and pass pointers



COMPRESS
Strip verbose tool outputs and keep the signal



ISOLATE
Use separate context for each sub-agent



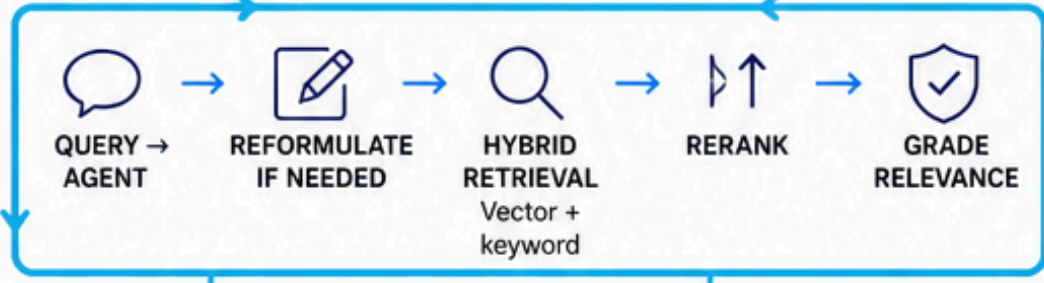
RETRIEVE
Pull only what this step needs

**KEY INSIGHT**

Context engineering is often more important than prompt wording.

Curating what the model sees changes what it can do.

4. AGENTIC RAG



QUERY → REFORMULATE IF NEEDED → HYBRID RETRIEVAL (Vector + keyword) → RERANK → GRADE RELEVANCE

IF EVIDENCE IS INSUFFICIENT
Return to AGENT → Reformulate → Retrieve again

IF EVIDENCE IS SUFFICIENT
GENERATE + CITE → VERIFY CLAIMS → ANSWER

NAIVE RAG
Retrieve once → Generate without checking

5. RETRIEVAL BUILDING BLOCKS

**CHUNKING**
Semantic usually beats fixed-size

**EMBEDDINGS**
Represent meaning for similarity

**HYBRID SEARCH**
BM25 + vector

**RERANKER**
Sort the best evidence

**METADATA FILTERS**
Scope by source, date, tenant, or access

**KNOWLEDGE GRAPHS**
Use explicit relationships

**CITATIONS**
Link claims to evidence

6. WHAT SHOULD MEMORY KEEP?

**KEEP**

- Stable preferences
- Verified facts with provenance
- Decisions and reusable outcomes

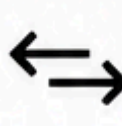
**SKIP OR EXPIRE**

- Secrets and sensitive data
- Transient status
- Unverified guesses
- Stale facts without timestamps

7. FAILURE MODES

**CONTEXT POISONING**
A bad fact persists

**CONTEXT DISTRACTION**
Too much noise hides the signal

**CONTEXT CLASH**
Sources contradict each other

**LOST IN THE MIDDLE**
Important details get overlooked

**STALE MEMORY**
Old facts are treated as current

**TAKEAWAY**

Retrieve less, but better.
Inside a limited context window, precision often beats recall.



MULTI-AGENT SYSTEMS

6. MULTI-AGENT ORCHESTRATION

1. WHY MULTIPLE AGENTS?



SPECIALIZATION
Narrow expertise per role



SEPARATE CONTEXT
Less cross-task noise



PARALLELISM
Independent work at the same time



MODULAR OWNERSHIP
Clear responsibility by component

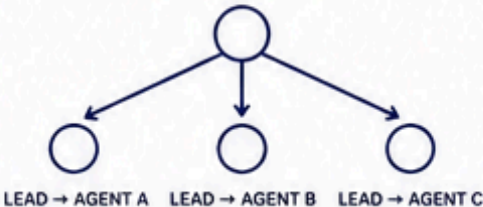


DISTINCT PERMISSIONS
Minimum tools and access per role



COORDINATION COST
More agents add overhead, error propagation, and debugging difficulty.

2. TOPOLOGY PATTERNS

SUPERVISOR

LEAD → AGENT A LEAD → AGENT B LEAD → AGENT C

Lead routes work.
Workers report back.

HIERARCHICAL

CEO

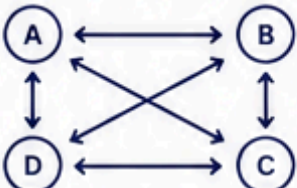
MANAGER 1 MANAGER 2

AGENTS A + B AGENTS C + D

PIPELINE

AGENT A AGENT B AGENT C

Assembly-line handoff

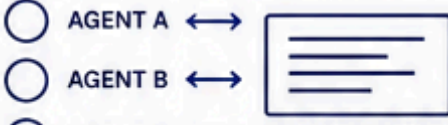
NETWORK / SWARM

Any-to-any handoff

DEBATE / ENSEMBLE

AGENT A AGENT B AGENT C

JUDGE ANSWER

BLACKBOARD

AGENT A AGENT B AGENT C

SHARED STATE

All agents read and write shared state.

3. THE SUPERVISOR FLOW MOST USED



4. ROLE DESIGN

Give each agent:



NARROW ROLE



MINIMUM TOOLS



OWN SYSTEM PROMPT



EXPLICIT OUTPUT CONTRACT



ESCAPE HATCH
"If out of scope, hand back"

5. COMMUNICATION



HANDOFF
Transfer control and relevant context



MESSAGE PASSING
Use structured JSON, not loose prose



SHARED MEMORY / BLACKBOARD
Maintain common state



A2A PROTOCOLS
Agent-to-agent interoperability across vendors

6. COORDINATION FAILURES



INFINITE PING-PONG
Agents bounce the task forever



CONTEXT LOSS
Critical details disappear at handoff



DUPLICATED WORK
Agents solve the same task twice



DIFFUSED RESPONSIBILITY
No owner is accountable



COMPOUNDING ERRORS
If each of five steps is 90% reliable:
 $0.9^5 \approx 0.59$ end-to-end reliability



DESIGN RULE
Define one owner for every deliverable.
Cap handoffs and retries.



TAKEAWAY

A single agent with great tools often beats a mediocre swarm.
Split work only when specialization, context, or permissions genuinely demand it.



7. THE ECOSYSTEM

1. FRAMEWORK LANDSCAPE

Framework	Model	Strength
 LangGraph	Graph / state machine	Control, checkpointing, HITL
 CrewAI	Role-based crews	Fast to prototype
 AutoGen	Conversational agents	Multi-agent chat, code execution
 OpenAI Agents SDK	Handoffs + guardrails	Lightweight, native tools
 Claude Agent SDK	Loop + tools + MCP	Long-running coding agents
 LlamaIndex	Data-centric	Agentic RAG
 Semantic Kernel	Enterprise plugins	.NET and Java shops
 Smolagents	Code-writing agents	Minimal, code as action

2. BUILD vs FRAMEWORK



SIMPLE LOOP + 3 TOOLS
Write it yourself
About 200 lines of code



BRANCHING + STATE + HUMAN REVIEW
Use a graph framework



MANY ROLES + HANDOFFS
Use a multi-agent framework



DESIGN TIP
Frameworks hide the loop.
Learn the loop first. You will debug it later either way.

3. PROTOCOL STACK



A2A
AGENT ↔ AGENT
Discovery and task delegation



MCP
AGENT ↔ TOOLS / DATA
Standardized connectors



FUNCTION CALLING
MODEL → JSON SCHEMA
TOOL INVOCATION

4. AGENT ARCHETYPES IN THE WILD



CODING AGENT
Repository context → Edit → Test → Fix



DEEP RESEARCH AGENT
Plan → Parallel search → Synthesize → Cite



COMPUTER-USE AGENT
Screenshot → Click or type → Verify



SUPPORT AGENT
RAG + CRM tools → Escalate when needed



DATA AGENT
Natural language → SQL → Chart → Narrative



OPS / SRE AGENT
Alert → Diagnose → Runbook → Remediate



COMMERCE AGENT
Browse → Compare → Checkout with approval

5. DEEP RESEARCH FLOW

QUESTION → CLARIFY → PLAN SUB-QUESTIONS

PLAN SEARCH A → PLAN SEARCH B → PLAN SEARCH C → PLAN SEARCH D

SEARCH A + B + C + D → DEDUPLICATE + RANK SOURCES → SYNTHESIZE → VERIFY CLAIMS → CITED REPORT

QUALITY CHECKS

- Source authority
- Date and freshness
- Independent corroboration
- Citation coverage

EVALUATION, OBSERVABILITY & COST

8. MAKING AGENTS MEASURABLE

1. WHAT TO MEASURE



OUTCOME
 Task success
 Goal completion
 Human acceptance

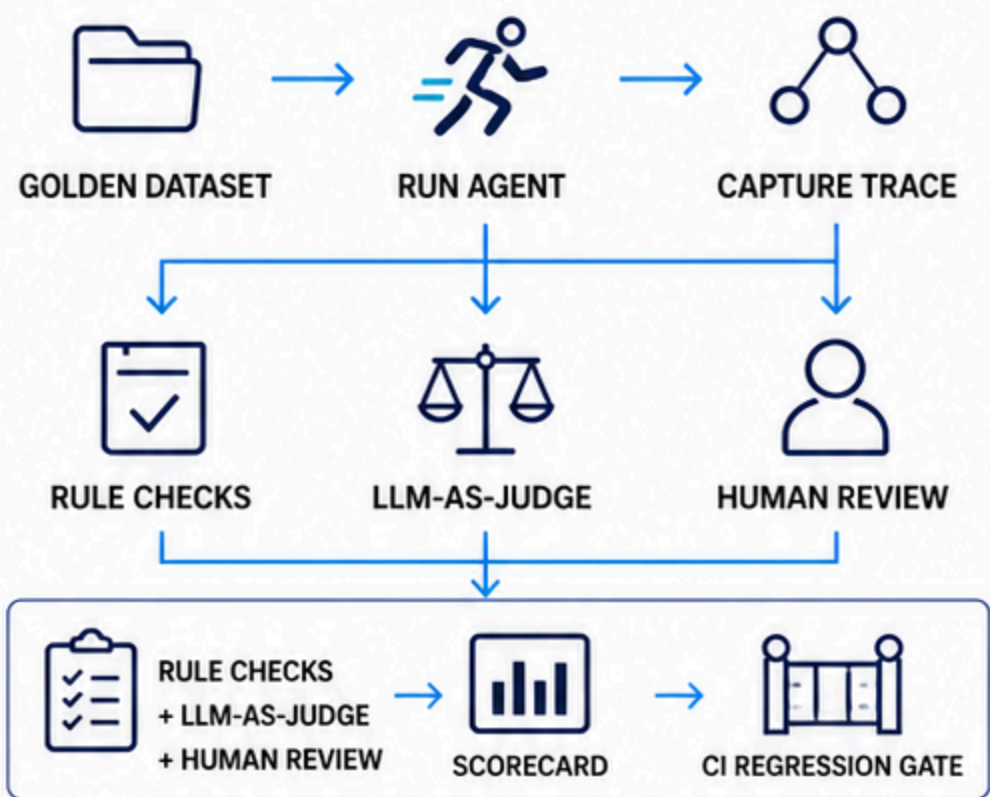


TRAJECTORY
 Correct tool
 Valid arguments
 Step efficiency
 Loop count



COMPONENT
 Retrieval precision@k
 Plan quality
 Format validity

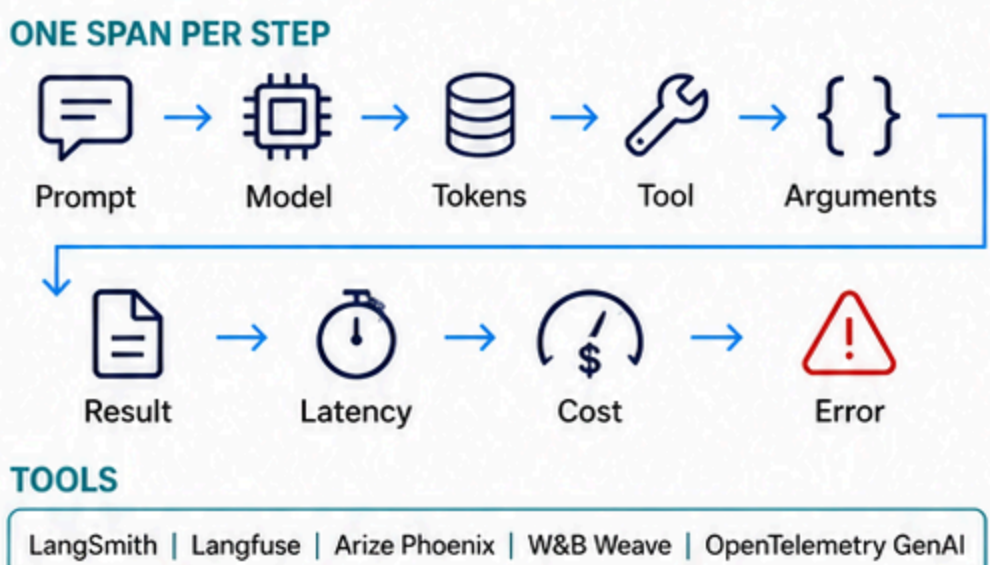
2. EVAL PIPELINE



3. KEY METRICS

Metric	Why it matters
Task success %	Goal achieved
Steps per task	Efficiency + cost
Tool error rate	Schema quality
Loop / retry rate	Reasoning stability
p50 / p95 latency	User experience
Cost per task	Unit economics
Escalation rate	Autonomy readiness

4. OBSERVABILITY | TRACE EVERYTHING



DEBUGGING RULE
 No replay, no reliable fix.

5. COST & LATENCY LEVERS



MODEL ROUTING
 Use small models first



PROMPT CACHING
 Cache static prompts



PARALLEL READS
 Run independent reads together



TRIM CONTEXT
 Keep only useful context



RESULT CACHING
 Reuse deterministic results



HARD LIMITS
 Cap steps and tokens

6. FAILURE TAXONOMY



Wrong plan



Wrong tool



Bad arguments



Ignored observation



Hallucinated result



Infinite loop



Context overflow



Premature stop



Silent partial failure



TAKEAWAY

Ship an evaluation set before you ship the agent.
 Intuition alone does not survive production.

SAFETY, GUARDRAILS & PRODUCTION

9. TRUST, SAFETY & DEPLOYMENT | 10. END-TO-END RECAP

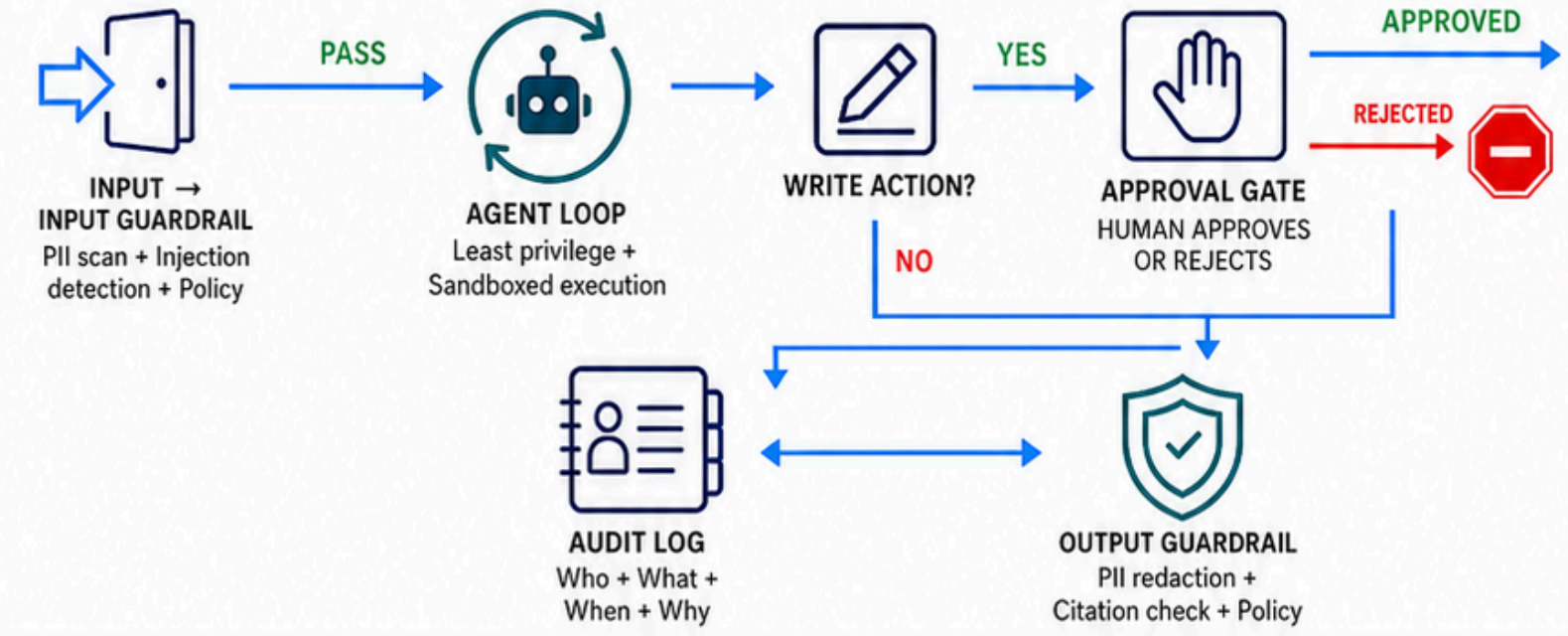
1. THE THREAT MODEL



**2. THE GOLDEN RULE**

Everything the agent reads is DATA, not INSTRUCTIONS.
Only the user through a trusted channel can issue commands.

3. DEFENSE-IN-DEPTH FLOW



4. GUARDRAIL TYPES



5. HUMAN-IN-THE-LOOP MODES



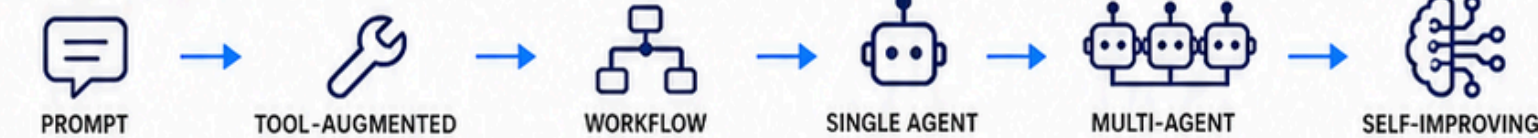
6. PRODUCTION READINESS

- ✓ Evaluation suite + Regression gate
- ✓ Full tracing
- ✓ Step, cost, and time caps
- ✓ Least-privilege credentials
- ✓ Approval gates on writes
- ✓ Idempotency keys
- ✓ Graceful degradation
- ✓ Audit log
- ✓ Rollback plan
- ✓ Kill switch

7. END-TO-END FLOW



8. MATURITY PATH



**TAKEAWAY**

Give a model a goal, tools, memory, and a bounded loop.
Then measure it, constrain it, and keep a human on the brake.